DBrain: Distributed Spiking Neural Network Simulation on a Self-Organizing Wireless Cluster of Commodity Microcontrollers

Ali Akrami

dibachain.ir

Abstract

The human brain performs its computation on roughly twenty watts; the machines we build to imitate it fill server racks. Between these extremes lies an almost unexplored middle ground: whether meaningful spiking neural network (SNN) simulation can be distributed across the cheapest computers made today — Wi-Fi microcontrollers — connected by nothing but the air between them. This paper presents DBrain, a system architecture for simulating spiking neural networks on self-organizing clusters of ESP32-family and ESP8266 microcontrollers, communicating exclusively over ESP-NOW, Espressif's connectionless 802.11 protocol. We derive the design from first principles and primary measurements: the Izhikevich neuron model in s16.15 fixed-point arithmetic for bit-identical behavior across heterogeneous chips; spike transport as Address Event Representation bundles carried by broadcast frames, treated as hardware multicast; a loss policy of counting rather than retransmitting, with biological synaptic delays absorbing radio latency; sub-tick clock synchronization built on hardware receive timestamps; and TDMA transmission slotting enabled by that synchronization. A capacity analysis against measured channel behavior indicates that a 40-board cluster sustains approximately 4×10^4 neurons and 10^7 synapses at biological firing rates — about one third the neuron count of a fruit fly and two orders of magnitude beyond the nervous system of Caenorhabditis elegans, whose complete connectome constitutes a natural first milestone. To our knowledge no prior system, hobbyist or academic, has exchanged spikes between microcontrollers over radio. We argue the approach is feasible, quantify its limits honestly, and outline a staged implementation path.

Keywords: spiking neural networks, neuromorphic computing, distributed simulation, microcontrollers, ESP32, ESP-NOW, wireless sensor networks, self-organization

1. Introduction

The most complex object known to science runs on the power budget of a dim light bulb. Eighty-six billion neurons, wired through a hundred trillion synapses, produce thought, memory, and motion while consuming about twenty watts [1]. The machines built to imitate that trick occupy the opposite corner of the design space: SpiNNaker fills a machine room with a million ARM cores [2]; Intel's Hala Point packs 1,152 Loihi 2 chips and draws 2.6 kW [3]. Both are magnificent, and both are unreachable — physically and financially — for almost everyone who might want to hold a spiking network in their hands.

This paper asks a deliberately modest question with a surprisingly rich answer: how far can spike-level brain simulation be pushed on the cheapest, most abundant computers ever manufactured? The ESP32 family of Wi-Fi microcontrollers sells for a few dollars per board, and hundreds of millions of units ship per year. Each has one or two cores in the 100–240 MHz range, a few hundred kilobytes of RAM, and — decisively — a radio. Espressif's ESP-NOW protocol lets these chips exchange frames directly, with no access point, no TCP/IP stack, and no infrastructure at all: two boards powered anywhere near each other can talk.

DBrain is a system architecture that turns an arbitrary collection of such boards into a single simulated nervous system. Each board owns a shard of neurons and synapses. When a neuron fires, the event crosses the air as part of a broadcast frame and is delivered into synapses on every board that subscribes to it. The cluster is self-organizing: boards discover each other, elect a coordinator, agree on time, and accept a share of the network proportional to their capability. Unplugging a board injures the brain; powering a new one grows it.

The contribution of this paper is threefold:

1. **A feasibility study grounded in primary sources and measurements** (Section 3): the exact payload, throughput, latency, and timing properties of ESP-NOW across the chip family, and the memory and arithmetic budgets of each variant.
2. **A complete system design** (Section 4) that transfers the architectural lessons of large neuromorphic machines — source-addressed multicast spikes, logical time, drop-don't-retry congestion policy, event-driven synapses — onto a broadcast radio medium, together with the distributed-systems machinery (membership, election, placement, synchronization) that a wireless cluster additionally requires.

3. **An honest capacity analysis** (Section 5) locating the achievable scale precisely on the biological axis: well beyond the complete nervous system of C. elegans, within reach of insect-brain subcircuits, and eight orders of magnitude short of a human brain — a gap we quantify rather than obscure.

A systematic search across academic literature and maker communities found no prior system in which microcontrollers exchange neural spikes over radio (Section 2.3). DBrain therefore occupies a genuinely open point in the design space: the architecture of the room-sized machines, at three dollars per node, over the air.

2. Related Work

2.1 Large-scale neuromorphic machines

SpiNNaker [2, 4] remains the reference architecture for real-time SNN simulation on programmable cores, and most of DBrain's load-bearing decisions descend from it. Its million ARM968 cores each simulate on the order of a thousand simple neurons on a 1 ms wall-clock tick. A spike is a 40-bit packet carrying only the identity of the neuron that fired; router tables compiled offline replicate each packet toward every subscribing core. Three of its choices matter here. First, there is no global barrier and no global clock: cores that overrun a busy tick catch up in quiet ones. Second, congested routers drop packets rather than retry, on the grounds that biological computation tolerates sparse spike loss but not stalled real time [4]. Third, neuron state lives in fast local memory while the far larger synaptic matrix resides in off-chip SDRAM, streamed only when a spike arrives [5]. It is worth noting that SpiNNaker 2 moved to Cortex-M4F cores at a few hundred megahertz [6] — silicon of the same class as the microcontrollers considered here.

Intel Loihi and Loihi 2 advance time by barrier synchronization events rather than a clock — a timestep is a logical agreement among cores [3, 7]. **IBM TrueNorth** runs a global 1 ms tick and stamps each spike packet with a small delivery-tick field so that a jittery transport can still deliver events at the correct logical time [8]. Both ideas transfer directly to a radio medium. **BrainScaleS** [9] takes the opposite pole — analog neurons running 10^3–$10^4\times$ faster than biology — and serves here mainly as a contrast.

2.2 Small systems and microcontroller projects

At the opposite end of the scale, Spikeling implements one Izhikevich neuron on a single ESP32 as a teaching instrument, with units interconnected by audio cables [10] — evidence that real-time Izhikevich dynamics are comfortable on this silicon. NeuroBytes produced physical PCB neurons joined by cables; the project was NSF-

supported, crowdfunded, shipped kits, and reconstructed a flatworm connectome before folding around 2020 [11]. Its trajectory demonstrates both the appeal of tangible neurons and the scaling ceiling of wired interconnect. The Lu.i project continues this niche with an analog leaky integrate-and-fire PCB [12]. General-purpose ESP32 clusters have been assembled — notably the Broccoli task-queue cluster [13] — and abandoned once demonstrated, with the network as the bottleneck and no compelling workload. Recent academic work has partitioned conventional (non-spiking) neural network inference layer-wise across wireless microcontrollers [14], the nearest published relative to the present proposal.

2.3 The gap

Searches across arXiv, IEEE, ACM, GitHub, Hackaday, and related venues found single-chip neuron instruments, wired neuron boards, generic microcontroller clusters, and rack-scale neuromorphic machines — but no system exchanging spikes between microcontrollers over any radio protocol. The documented failure modes of the neighbors are instructive: flooding meshes collapse under dense traffic [15]; JSON-per-message transports drown in overhead [16]; central-master orchestration without a compelling demonstration precedes abandonment [13]. The design below is shaped by those autopsies.

3. Background and Constraints

Every architectural decision in Section 4 is forced by a measured or documented constraint. This section fixes those constraints.

3.1 ESP-NOW

ESP-NOW transmits vendor-specific IEEE 802.11 action frames, connectionless and infrastructure-free [17]. The version 1 payload limit — and the limit for any frame an ESP8266 must receive — is 250 bytes; version 2 (ESP-IDF ≥ 5.4) extends this to 1,470 bytes for newer chips, with v1 devices unable to receive larger frames. The peer table holds at most 20 entries, but broadcast frames require no peer registration and reach an unlimited audience — the property on which DBrain's transport rests. Broadcast frames receive no MAC-layer acknowledgment and no retransmission; unicast frames are MAC-acknowledged and retried, though a receiver whose buffer overflowed still acknowledges, so link-layer "success" does not guarantee application delivery [18].

Measured behavior fixes the performance envelope. At the default 1 Mbps PHY rate, independent benchmarks report approximately 90–100 kB/s of application throughput and roughly 345–404 maximum-size frames per second on a saturated channel [19];

raising the rate to 24–54 Mbps (unavailable to ESP8266) yields 780–960 kB/s for broadcast [19]. End-to-end latency for small frames is a median 5.6 ms including the software path [20]. Two timing measurements are architecturally decisive: a single broadcast reaches multiple receivers within a spread of only ~80 µs, while the sender's own transmit instant jitters by up to 5 ms under CSMA and task scheduling [21]. Synchronization must therefore be built on reception, never transmission. ESP-IDF ≥ 5.1 exposes a microsecond-resolution hardware receive timestamp in the receive callback [17], providing exactly the primitive required; the 802.11 TSF timer, by contrast, does not run without an access-point association and is unusable here [22].

3.2 The chip family

Verified against vendor documentation [23]: the ESP32, S2, S3, C2, C3, C5, and C6 all carry 2.4 GHz Wi-Fi and support ESP-NOW, with 272–520 KB of SRAM and one or two cores at 120–240 MHz. The ESP8266 participates with v1 frames only, ~40–50 KB of usable heap, and no hardware receive timestamp. Two family members are excluded by physics rather than software: the ESP32-H2 has no Wi-Fi radio at all, and the ESP32-P4 has no radio and requires a companion chip. Critically, most of the family — including, counterintuitively, the S2 — has no floating-point unit; on FPU-less chips a software float addition costs ~100 cycles and a cosine ~2,400 [24]. Optional external PSRAM (4–8 MB on common ESP32/S3 modules) is an order of magnitude slower than internal SRAM but tolerates sequential streaming well.

3.3 Neuron model economics

Izhikevich's canonical comparison [25] prices the candidate models: a leaky integrate-and-fire neuron costs ~5 operations per simulated millisecond but reproduces only 3 of ~20 catalogued biological firing behaviors; the Hodgkin–Huxley model reproduces all of them at ~1,200 operations; the Izhikevich model reproduces all 20 at 13 operations with two state variables. Cortical firing rates follow a lognormal distribution spanning roughly 0.1–10 Hz with means of a few hertz [26] — a sparsity that makes event-driven synaptic processing, and by extension radio-linked simulation, feasible at all.

4. System Design

4.1 Simulation core

Each node runs the hybrid loop that real-time SNN engines have converged on [5]: time-driven neuron integration on a fixed tick (1 ms of biological time), with synaptic work performed only on spike arrival. Incoming spikes index a per-source synaptic row;

each traversed synapse adds its weight into the target neuron's input accumulator at `now + delay`, implemented as a ring of per-tick buckets.

All arithmetic is integer fixed-point in the s16.15 format — the same format used, with published accuracy analyses, throughout SpiNNaker's production software [5, 27]. This choice is forced twice over: most supported chips have no FPU (Section 3.2), and a heterogeneous cluster must produce bit-identical results wherever a neuron is placed, so that placement is invisible to the simulation and any node's behavior can be reproduced exactly on a host-side reference simulator. Synapses pack into 32 bits — 16-bit weight, 4-bit delay, 8-bit local target index, type and padding — following SpiNNaker's layout [5]. Neuron state occupies 8 bytes. On an ESP32 with 4 MB of PSRAM, neuron state resides in internal SRAM while synaptic rows stream from PSRAM on spike arrival, mirroring SpiNNaker's SDRAM split; the budget supports on the order of 10^3 neurons and 10^6 synapses per board (Section 5).

4.2 Spike transport

Spikes cross board boundaries in Address Event Representation [28]: a packet identifies which neuron fired and nothing else. DBrain's key observation is that an ESP-NOW broadcast **is** hardware multicast: one transmitted frame is received by every node in the cell. Where SpiNNaker replicates packets through compiled router tables, DBrain broadcasts each spike bundle once; each receiving node binary-searches the source identifier against a local subscription table — the software analog of SpiNNaker's ternary CAM — and either processes or discards the bundle. No routing tables, no forwarding, no per-peer state, and no dependence on the 20-peer limit. A spike consumed by seven boards costs one frame.

Fired spikes accumulate during a tick and transmit as a single bundle of up to 119 two-byte events in a 250-byte frame (12-byte header carrying source node, tick number, and a per-node sequence number). Version 1 framing is retained for all simulation traffic so that every supported chip can receive it.

The loss policy is imported deliberately from SpiNNaker [4]: spike bundles are fire-and-forget; sequence-number gaps are counted and reported, never repaired. A retransmitted spike is a late spike, and a late spike is either wrong or a stall. Two mechanisms make this safe. First, biological synaptic delays (1–15 ms; the 4-bit delay field) absorb transport latency exactly as TrueNorth's delivery-tick field does [8]: a bundle stamped tick T arriving during tick T+1 still applies every synapse of delay ≥ 2 at its correct biological time. The network compiler enforces a minimum cross-board delay of two ticks, rendering ordinary radio jitter invisible to the simulation. Second,

control-plane traffic — join, assignment, migration — uses the opposite contract: application-level acknowledgment with bounded retry over unicast.

4.3 Time

Logical time advances in ticks; the wall-clock duration of one tick is a cluster constant (the slowdown factor), chosen from the airtime budget — unity for small, fast clusters; 5–10× for large or ESP8266-inclusive ones. There is no per-tick barrier: as on SpiNNaker, an overrunning node catches up during quieter ticks, and the delay slack of Section 4.2 hides transient lag. A paced mode with explicit per-tick acknowledgment exists for hard-reproducibility experiments at the price of running at the speed of the slowest node.

Synchronization exploits the measured asymmetry of Section 3.1. The coordinator broadcasts a beacon each second carrying the tick count and its own clock; every receiver pairs the beacon with its hardware receive timestamp and maintains an eight-point linear regression of coordinator time against local time, slewing — never stepping — its tick phase. Crystal tolerances of ±10 ppm [29] bound inter-node drift at 20 µs/s, so residual error remains far inside a tick between beacons, and the regression suppresses crystal skew to the ~1 ppm class, as in FTSP [30]. The realistic accuracy floor is set by the ~80 µs receiver-side spread; we target < 100 µs cluster-wide and note candidly that published claims of tighter synchronization over ESP-NOW lack rigorous measurement [31]. One hundred microseconds is one tenth of a tick — sufficient with margin for the mechanism that synchronization exists to serve: once nodes share time, each tick's transmission window is divided into assigned TDMA slots, converting synchronized burst collisions — the broadcast channel's worst enemy, unprotected by acknowledgments and aggravated by hidden nodes [32] — into scheduled, collision-free airtime. Synchronization enables slotting; slotting protects the beacons that maintain synchronization.

ESP8266 nodes, lacking the hardware timestamp, synchronize at millisecond precision in software and receive proportionally wider guard bands.

4.4 Self-organization

Membership. All membership state derives from 1 Hz jittered broadcast heartbeats carrying each node's identity, capability record (chip, cores, clock, free memory, PSRAM, FPU), epoch, and health counters. On a broadcast medium every node overhears every heartbeat, so passive listening replaces most of an active gossip protocol; a hundred-node table costs ~2.4 KB. Failure detection follows SWIM [33]: missed heartbeats mark a node suspect, indirect probes through two peers guard

against asymmetric radio links, and confirmed death triggers reassignment. A new board broadcasts join beacons across channels until it hears the cluster, then receives its identity and, if resources allow, a shard.

Coordination. The coordinator is deliberately not a master: the spike plane is peer-to-peer and continues — degraded — with no coordinator at all. Its duties are the slow path only: beacons, admission, placement, epoch numbering. Election is therefore the simplest mechanism that works, essentially FTSP root election [30]: the highest (capability score ‖ MAC) key wins; a node hearing no beacon for a timeout self-promotes with an incremented epoch; higher keys and epochs preempt immediately. Partitions heal deterministically on merge, and split-brain during a partition costs only counted spike loss across the split. Raft-class consensus was evaluated and rejected: it presumes reliable ordered unicast and purchases a replicated log that a distinguished beacon source plus an epoch counter renders unnecessary [34].

Placement. Assigning neurons to boards is graph partitioning under a multicast cost model: an off-board source neuron costs one expected frame per firing regardless of how many synapses subscribe on a given board, so the objective counts unique (source, subscribing-board) pairs weighted by firing rate — a hypergraph formulation. SpiNeMap demonstrated 45% energy reductions on neuromorphic hardware by partitioning against measured rather than assumed spike rates [35]; DBrain adopts the same measure-and-refine loop through its telemetry. Shard sizing balances tick compute time, not neuron count, against each node's advertised capability, honoring the bulk-synchronous rule that the slowest node sets the pace [36]. A compilerless mode also exists: the coordinator broadcasts a generative recipe (topology family, parameters, seed) from which every node derives its own population and connectivity deterministically — the network as a pure function of the recipe, with no tables shipped at all.

5. Capacity Analysis and Biological Scale

5.1 Airtime

At 1 Mbps, a 250-byte broadcast occupies ≈ 2.5–2.9 ms of air including preamble, inter-frame spacing, and mean backoff — a saturation ceiling near 345 frames/s, consistent with measured 404 frames/s under ideal conditions [19]. Holding channel occupancy at a conservative 30% (broadcast collisions being unrecoverable), the cluster-wide budget is approximately 100 bundles/s at 1 Mbps and 600–1,150 bundles/s at 24–54 Mbps, i.e. roughly 1.2×10^4 and $0.7–1.4 \times 10^5$ spike events per second respectively at 119 events per bundle.

Demand scales as (nodes × neurons per node × mean rate). Forty boards of 1,000 neurons at a 3 Hz mean generate 1.2×10^5 events/s — at the upper edge of the high-rate budget, which TDMA slotting and rate-aware placement are designed to exploit fully. At a 1 Hz mean (well within the biological lognormal range [26]), the same budget carries on the order of a hundred boards. With ESP8266 nodes present, the 1 Mbps compatibility contract applies, and the cluster compensates with its slowdown factor — precisely the accommodation SpiNNaker's cortical-microcircuit work made in its first release before later optimization reached real time [37].

5.2 Where this lands on the biological axis

The comparison below uses accepted reference values: 86×10^9 neurons for the human brain [1], the complete adult Drosophila connectome of 139,255 neurons and 54.5×10^6 synapses published in 2024 [38], and the 302-neuron, ~7,600-connection nervous system of C. elegans [39].

System	Neurons	Synapses	Power
Human brain	8.6×10^{10}	$\sim 10^{14}$–10^{15}	~20 W
Fruit fly (adult, complete)	1.39×10^5	5.45×10^7	$\sim 10^{-4}$ W
C. elegans	3.02×10^2	$\sim 7.6 \times 10^3$	—
One SpiNNaker chip	$\sim 10^4$	$\sim 10^6$–10^7	~1 W
DBrain, 40 boards	**$\sim 4 \times 10^4$**	**$\sim 10^7$**	**~20–40 W**

Three readings of this table frame the project honestly.

Against the human brain, the gap is a factor of $\sim 2 \times 10^6$ in neurons and $\sim 10^7$ in synapses. Closing it with this hardware would require ~86 million boards, a quarter of a billion dollars of silicon, and tens of megawatts — against the brain's twenty watts. No incremental engineering closes that gap; we state it as a boundary, not a goal.

Against the small end of biology, the position is far more interesting. A 40-board cluster exceeds the complete nervous system of C. elegans by two orders of magnitude — an animal that locomotes, forages, avoids, and learns with those 302 neurons. The full worm connectome fits on two to three boards with capacity to spare, making it the natural first biological milestone: a physically distributed, radio-linked execution of a real organism's entire neural wiring, driving robotic behavior. At the cluster's upper end, $\sim 4 \times 10^4$ neurons is roughly 30% of a fruit fly's brain — the scale of well-characterized insect subcircuits such as the olfactory pathway.

Against the machines, a 40-board cluster is approximately one SpiNNaker chip in neuron count. That is the honest exchange rate between a wired backplane and the open air — and the cluster costs two orders of magnitude less than a SpiNNaker board, requires no infrastructure, and grows by powering another node.

6. Discussion and Limitations

The channel is the machine. Every scaling property of the system reduces to airtime arithmetic. The mitigations — batching, TDMA, multicast-aware placement, the slowdown factor — are all in the design, but a 2.4 GHz environment shared with domestic Wi-Fi will erode the budget unpredictably; the design makes this erosion visible through telemetry rather than pretending to prevent it.

The synchronization floor is asserted, not yet measured. The < 100 µs target rests on the documented behavior of hardware receive timestamps and one published receiver-spread measurement [21]; no rigorous multi-variant measurement of ESP-NOW synchronization exists in the literature. The implementation plan places this measurement (GPIO and logic analyzer) before any component that depends on it.

Heterogeneity has a price. ESP8266 participation constrains audible frames to 250 bytes at 1 Mbps and lowers the whole cluster's ceiling; the architecture confines this cost to clusters that actually include such nodes. Two family members (H2, P4) cannot participate for want of a radio — a physical boundary worth stating plainly.

Security is deferred by design. Broadcast frames cannot be encrypted at the protocol layer, and per-peer encryption is incompatible with broadcast-first operation; version 1 is an open laboratory instrument. An application-layer authenticated-encryption path (cluster-wide key, ECDH provisioning) is specified but unimplemented.

Simulation loss semantics. Spike loss is counted, bounded by design, and biologically defensible — but it is loss; experiments requiring exact reproducibility must use the paced mode and accept its speed, or replay on the bit-identical host reference simulator.

7. Roadmap

The implementation proceeds in phases, each terminating in a running artifact and a measured number: (0) a transport benchmark suite replacing this paper's borrowed channel figures with measurements on the actual fleet; (1) two boards exchanging a synfire chain over the air; (2) synchronization and TDMA, with the sync residual measured by logic analyzer; (3) live membership — join, election, failure, recovery —

under a running raster; (4) the network compiler and bit-identical host reference; (5) the gateway and browser dashboard with live rasters and telemetry; (6) STDP plasticity, whose deferred event-driven formulation transfers from SpiNNaker unchanged because pre-synaptic spikes arrive at the board that owns the synapse [40]. The biological milestone — the complete C. elegans connectome driving a wheeled robot — follows Phase 4.

8. Conclusion

The large neuromorphic machines answered the question of whether spiking brains can be simulated in real time on programmable silicon. DBrain asks the complementary question: how much of that answer survives when the backplane is replaced by air and the budget by pocket money. The analysis here says: enough to matter. The protocol limits, the arithmetic, the memory, and the measured timing of commodity Wi-Fi microcontrollers jointly admit a self-organizing, radio-linked spiking network of tens of thousands of neurons — beyond a complete biological nervous system, within sight of insect subcircuits, and honest about everything it is not. The remaining questions are not conceptual but empirical, and the roadmap is built to answer them in order.

References

[1] S. Herculano-Houzel, "The human brain in numbers: a linearly scaled-up primate brain," Frontiers in Human Neuroscience, vol. 3, no. 31, 2009.

[2] S. B. Furber, F. Galluppi, S. Temple, and L. A. Plana, "The SpiNNaker Project," Proceedings of the IEEE, vol. 102, no. 5, pp. 652–665, 2014.

[3] M. Davies et al., "Loihi: A Neuromorphic Manycore Processor with On-Chip Learning," IEEE Micro, vol. 38, no. 1, pp. 82–99, 2018.

[4] A. G. D. Rowley et al., "SpiNNTools: The Execution Engine for the SpiNNaker Platform," Frontiers in Neuroscience, vol. 13, 2019.

[5] O. Rhodes et al., "sPyNNaker: A Software Package for Running PyNN Simulations on SpiNNaker," Frontiers in Neuroscience, vol. 12, 2018.

[6] C. Mayr, S. Höppner, and S. Furber, "SpiNNaker 2: A 10 Million Core Processor System for Brain Simulation and Machine Learning," arXiv:1911.02385, 2019.

[7] Intel Corporation, "Taking Neuromorphic Computing to the Next Level with Loihi 2," technology brief, 2021.

[8] F. Akopyan et al., "TrueNorth: Design and Tool Flow of a 65 mW 1 Million Neuron Programmable Neurosynaptic Chip," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 34, no. 10, pp. 1537–1557, 2015.

[9] C. Pehle et al., "The BrainScaleS-2 Accelerated Neuromorphic System with Hybrid Plasticity," Frontiers in Neuroscience, vol. 16, 2022.

[10] A. M. Chagas et al., "Spikeling: A low-cost hardware implementation of a spiking neuron for neuroscience teaching and outreach," PLOS Biology, vol. 16, no. 10, 2018.

[11] NeuroTinker LLC, "NeuroBytes: Electronic Neuron Simulators," project documentation, hackaday.io/project/3339, 2014–2020.

[12] S. Billaudelle, J. Göltz, and Y. Stradmann, "Lu.i: an educational printed-circuit neuron," Trends in Neuroscience and Education, 2025.

[13] W. Lin, "Broccoli: Distributed task queue for ESP32 clusters," github.com/Wei1234c/Broccoli, 2018.

[14] "Decentralized Distributed Sequential Neural Network Inference on Low-Power Microcontrollers in Wireless Sensor Networks," Sensors, vol. 25, no. 15, art. 4595, 2025.

[15] A. Pii, "EspNowFloodingMesh," github.com/arttupii/EspNowFloodingMesh, 2019.

[16] painlessMesh contributors, "painlessMesh," gitlab.com/painlessMesh/painlessMesh.

[17] Espressif Systems, "ESP-NOW," ESP-IDF Programming Guide, docs.espressif.com/projects/esp-idf, api-reference/network/esp_now.

[18] MicroPython contributors, "espnow — support for the ESP-NOW protocol," docs.micropython.org/en/latest/library/espnow.html.

[19] Nanyang Technological University PS9888 group, "ESP-NOW data transfer rate measurements," blogs.ntu.edu.sg/ps9888-2020-g16, 2020.

[20] Electric UI, "Latency comparison of common wireless links for microcontrollers," electricui.com/blog/latency-comparison, 2024.

[21] "ESP-NOW broadcast transmission latency," measurement thread, forum.arduino.cc, 2024.

[22] Espressif Systems, GitHub issue #7244, "esp_wifi_get_tsf_time execution time," github.com/espressif/esp-idf, 2021.

[23] Espressif Systems, product pages for ESP32, ESP32-S2/S3, ESP32-C2/C3/C5/C6, ESP32-H2, ESP32-P4, espressif.com/en/products/socs.

[24] Espressif Systems Developer Portal, "Which ESP32 chips have a floating-point unit?," developer.espressif.com/blog, 2025.

[25] E. M. Izhikevich, "Which Model to Use for Cortical Spiking Neurons?," IEEE Transactions on Neural Networks, vol. 15, no. 5, pp. 1063–1070, 2004.

[26] G. Buzsáki and K. Mizuseki, "The log-dynamic brain: how skewed distributions affect network operations," Nature Reviews Neuroscience, vol. 15, pp. 264–278, 2014.

[27] M. Hopkins and S. Furber, "Accuracy and Efficiency in Fixed-Point Neural ODE Solvers," Neural Computation, vol. 27, no. 10, pp. 2148–2182, 2015.

[28] K. A. Boahen, "Point-to-point connectivity between neuromorphic chips using address events," IEEE Transactions on Circuits and Systems II, vol. 47, no. 5, pp. 416–434, 2000.

[29] Espressif Systems, ESP Hardware Design Guidelines, docs.espressif.com/projects/esp-hardware-design-guidelines.

[30] M. Maróti, B. Kusy, G. Simon, and Á. Lédeczi, "The Flooding Time Synchronization Protocol," in Proc. ACM SenSys, pp. 39–49, 2004.

[31] "Time Synchronization on ESP-NOW Based Wireless Sensor Network Using Flooding Time Synchronization Protocol," in Proc. SIET, ACM, 2023.

[32] "Hidden node problem," in Computer Science Topics, ScienceDirect reference collection.

[33] A. Das, I. Gupta, and A. Motivala, "SWIM: Scalable Weakly-consistent Infection-style Process Group Membership Protocol," in Proc. IEEE DSN, pp. 303–312, 2002.

[34] M. A. Rahman, "Leader Election in the Internet of Things: Challenges and Solutions," arXiv:1911.00759, 2019.

[35] A. Balaji et al., "Mapping Spiking Neural Networks to Neuromorphic Hardware," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 28, no. 1, pp. 76–86, 2020.

[36] L. G. Valiant, "A bridging model for parallel computation," Communications of the ACM, vol. 33, no. 8, pp. 103–111, 1990.

[37] S. J. van Albada et al., "Performance Comparison of the Digital Neuromorphic Hardware SpiNNaker and the Neural Network Simulation Software NEST for a Full-Scale Cortical Microcircuit Model," Frontiers in Neuroscience, vol. 12, 2018.

[38] S. Dorkenwald et al., "Neuronal wiring diagram of an adult brain," Nature, vol. 634, pp. 124–138, 2024.

[39] J. G. White, E. Southgate, J. N. Thomson, and S. Brenner, "The structure of the nervous system of the nematode Caenorhabditis elegans," Philosophical Transactions of the Royal Society B, vol. 314, pp. 1–340, 1986; S. J. Cook et al., "Whole-animal connectomes of both Caenorhabditis elegans sexes," Nature, vol. 571, pp. 63–71, 2019.

[40] P. U. Diehl and M. Cook, "Efficient Implementation of STDP Rules on SpiNNaker Neuromorphic Hardware," in Proc. IJCNN, pp. 4288–4295, 2014.